

数据库设计

1. ER 图

实体属性明细表

sys_user（用户表）

字段	类型	键	说明
id	BIGINT	PK	用户唯一标识
username	VARCHAR(50)	UK	登录账号
password_hash	VARCHAR(255)		密码
real_name	VARCHAR(100)		真实姓名
role	VARCHAR(20)		admin / creator / user
can_create_task	TINYINT		是否可创建任务
last_seen	DATETIME		最后活跃时间
status	TINYINT		1=在线 0=离线
is_deleted	TINYINT		0=正常 1=已删除

guide_version（指南版本表）

字段	类型	键	说明
id	INT	PK	版本ID
version_name	VARCHAR(100)		版本名称
description	TEXT		版本说明
created_at	DATETIME		创建时间
attachment_name	VARCHAR(255)		附件文件名

label_l1（一级标签表）

字段	类型	键	说明
id	INT	PK	一级标签ID
guide_version_id	INT	FK	所属指南版本
name	VARCHAR(50)		标签全称
abbr	VARCHAR(30)		标签简称

字段	类型	键	说明
description	TEXT		标签说明

label_l2（二级标签表）

字段	类型	键	说明
id	INT	PK	二级标签ID
guide_version_id	INT	FK	所属指南版本
parent_l1_id	INT	FK	所属一级标签
name	VARCHAR(50)		标签全称
abbr	VARCHAR(30)		标签简称
description	TEXT		标签说明

relation_type（关系类型表）

字段	类型	键	说明
id	INT	PK	关系类型ID
guide_version_id	INT	FK	所属指南版本
name	VARCHAR(50)		关系名称
abbr	VARCHAR(30)		关系简称
description	TEXT		关系说明
is_binary	TINYINT		是否二元关系

global_document（全局文书库表）

字段	类型	键	说明
id	BIGINT	PK	文书ID
title	VARCHAR(255)		文书标题
file_name	VARCHAR(255)		文件名
file_type	VARCHAR(255)		用户自定义文书类型
extracted_text	LONGTEXT		提取的纯文本
uploaded_at	DATETIME		上传时间

task（任务表）

字段	类型	键	说明
id	INT	PK	任务ID

字段	类型	键	说明
title	VARCHAR(100)		任务名称
description	TEXT		任务描述
status	ENUM		标注中 / 待裁定 / 可导出
creator_id	BIGINT	FK	创建者ID → sys_user.id
guide_version_id	INT	FK	指南版本ID → guide_version.id
created_at	DATETIME		创建时间
stage_changed_at	DATETIME		阶段变更时间

task_member（任务成员表）

字段	类型	键	说明
id	INT	PK	分配记录ID
task_id	INT	FK	任务ID → task.id
user_id	BIGINT	FK	用户ID → sys_user.id
role_in_task	ENUM		标注员 / 裁定者

task_document（任务文档表）

字段	类型	键	说明
id	INT	PK	文档关联ID
task_id	INT	FK	所属任务ID → task.id
source_type	ENUM		UPLOAD / RECREATE / GLOBAL
global_doc_id	BIGINT	FK	全局文书ID（可空）→ global_document.id
file_name	VARCHAR(255)		文件名
file_path	VARCHAR(500)		存储路径
extracted_text	LONGTEXT		纯文本内容
uploaded_at	DATETIME		添加时间
status	VARCHAR(20)		文书阶段状态

annotation（标注结果表）

字段	类型	键	说明
id	BIGINT	PK	标注记录ID
task_id	BIGINT	FK	任务ID → task.id

字段	类型	键	说明
document_id	BIGINT	FK	文档ID → task_document.id
user_id	BIGINT	FK	用户ID → sys_user.id
record_type	VARCHAR(20)		ANNOTATION / ARBITRATION
status	VARCHAR(20)		DRAFT / SUBMITTED
is_final	TINYINT		是否最终版本
guide_version_id	BIGINT		标注时指南版本
guide_snapshot	JSON		标签配置快照
submitted_at	DATETIME		提交时间
layout_json	JSON		论证图布局
reject_reason	VARCHAR(500)		退回原因
created_at	DATETIME		创建时间
updated_at	DATETIME		更新时间

proposition（命题表）

字段	类型	键	说明
id	BIGINT	PK	命题ID
annotation_id	BIGINT	FK	所属标注记录 → annotation.id
display_id	VARCHAR(20)	UK	展示ID（P1, P2...）
sequence_no	INT		文中序号
start_pos	INT		起始字符位置
end_pos	INT		结束字符位置
selected_text	TEXT		选中文本片段
label_l1	VARCHAR(10)		一级标签简称
label_l2	VARCHAR(30)		二级标签简称
label_path	VARCHAR(50)		标签路径
created_at	DATETIME		创建时间
updated_at	DATETIME		更新时间

argument_relation（关系表）

字段	类型	键	说明
id	BIGINT	PK	关系ID

字段	类型	键	说明
annotation_id	BIGINT	FK	所属标注记录 → annotation.id
display_id	VARCHAR(20)	UK	展示ID (R1, R2...)
sequence_no	INT		序号
relation_type	VARCHAR(10)		关系类型简称
expression	VARCHAR(500)		自然语言表达式
created_at	DATETIME		创建时间
updated_at	DATETIME		更新时间

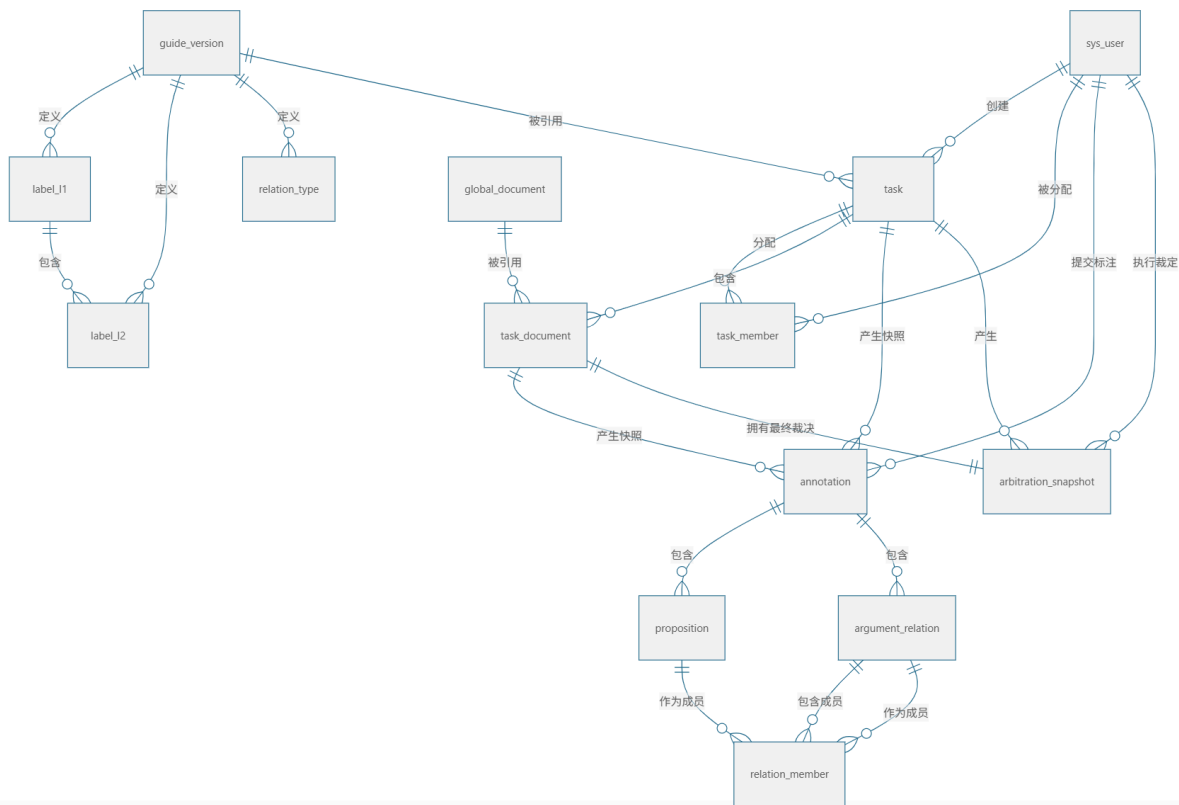
relation_member (关系成员明细表)

字段	类型	键	说明
id	BIGINT	PK	成员明细ID
relation_id	BIGINT	FK	所属关系ID → argument_relation.id
member_type	VARCHAR(1)		P=命题 / R=关系
proposition_id	BIGINT	FK	命题ID (member_type=P) → proposition.id
child_relation_id	BIGINT	FK	子关系ID (member_type=R) → argument_relation.id
member_role	VARCHAR(10)		SOURCE / TARGET / MEMBER
member_order	INT		参数顺序
created_at	DATETIME		创建时间

arbitration_snapshot (裁定快照表)

字段	类型	键	说明
id	INT	PK	裁定快照ID
task_id	INT	FK	任务ID → task.id
task_document_id	INT	FK	文档ID (唯一) → task_document.id
arbitrator_id	BIGINT	FK	裁定者ID → sys_user.id
adopted_from	VARCHAR(50)		采纳来源
final_result	TINYINT		最终结果标记
arbitrated_at	DATETIME		裁定时间

实体间关系



Mermaid 源文件: [P3-实体间关系.mmd](#)

各实体间关系明细

序号	源实体	目标实体	关系描述	基数	外键位置	备注
1	sys_user	task	创建	1:N	task.creator_id → sys_user.id	任务创建者
2	sys_user	task_member	被分配	1:N	task_member.user_id → sys_user.id	用户被分配为标注员/裁定者
3	sys_user	annotation	提交	1:N	annotation.user_id → sys_user.id	标注员提交标注
4	sys_user	arbitration_snapshot	执行	1:N	arbitration_snapshot.arbitrator_id → sys_user.id	裁定者执

序号	源实体	目标实体	关系描述	基数	外键位置	备注
						行裁决
5	guide_version	label_l1	定义	1:N	label_l1.guide_version_id → guide_version.id	版本包含一级标签
6	guide_version	label_l2	定义	1:N	label_l2.guide_version_id → guide_version.id	版本包含二级标签
7	guide_version	relation_type	定义	1:N	relation_type.guide_version_id → guide_version.id	版本包含关系类型
8	guide_version	task	被引用	1:N	task.guide_version_id → guide_version.id	一个版本可被多个任务使用
9	label_l1	label_l2	包含	1:N	label_l2.parent_l1_id → label_l1.id	一级标签包含二级标签
10	global_document	task_document	被引用	1:N	task_document.global_doc_id → global_document.id	全局文件被多个任务引用（可选）
11	task	task_member	分配	1:N	task_member.task_id → task.id	任务有多条分配记录

序号	源实体	目标实体	关系描述	基数	外键位置	备注
12	task	task_document	包含	1:N	task_document.task_id → task.id	任务包含多个数据条目
13	task	annotation	产生快照	1:N	annotation.task_id → task.id	任务有多个标注结果记录
14	task	arbitration_snapshot	产生	1:N	arbitration_snapshot.task_id → task.id	任务有多个文档的裁决结果
15	task_document	annotation	产生快照	1:N	annotation.document_id → task_document.id	文档有多个标注记录
16	task_document	arbitration_snapshot	拥有	1:1	arbitration_snapshot.task_document_id → task_document.id (UNIQUE)	每个文档仅一条最终裁决
17	annotation	proposition	包含	1:N	proposition.annotation_id → annotation.id	标注记录包含多个命题
18	annotation	argument_relation	包含	1:N	argument_relation.annotation_id → annotation.id	标注记录包含多个关系

序号	源实体	目标实体	关系描述	基数	外键位置	备注
19	argument_relation	relation_member	包含成员	1:N	relation_member.relation_id → argument_relation.id	关系对应多条成员记录
20	proposition	relation_member	作为成员	1:N	relation_member.proposition_id → proposition.id (member_type='P')	命题被多个关系引用
21	argument_relation	relation_member	作为成员	1:N	relation_member.child_relation_id → argument_relation.id (member_type='R')	关系支持嵌套

2. 建表 SQL

2.1 用户表

```
CREATE TABLE IF NOT EXISTS `sys_user` (
  `id` BIGINT UNSIGNED AUTO_INCREMENT COMMENT '用户唯一标识',
  `username` VARCHAR(50) NOT NULL COMMENT '登录账号',
  `password_hash` VARCHAR(255) NOT NULL COMMENT '密码（演示环境可为明文）',
  `real_name` VARCHAR(100) NOT NULL COMMENT '真实姓名',
  `role` VARCHAR(20) NOT NULL DEFAULT 'user' COMMENT 'admin / creator / user',
  `can_create_task` TINYINT NOT NULL DEFAULT 0 COMMENT '1=可创建任务',
  `last_seen` DATETIME DEFAULT NULL COMMENT '最后活跃时间',
  `status` TINYINT NOT NULL DEFAULT 0 COMMENT '1=在线 0=离线',
  `is_deleted` TINYINT NOT NULL DEFAULT 0 COMMENT '0=正常 1=已删除',
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_username` (`username`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户表';
```

说明：角色直接存储在 `role` 字段中（VARCHAR），不设独立的角色表。用户删除采用软删除（`is_deleted` 标记）。

2.2 指南版本表

```
CREATE TABLE IF NOT EXISTS `guide_version` (
  `id` INT UNSIGNED AUTO_INCREMENT,
```

```

`version_name` VARCHAR(100) NOT NULL,
`description` TEXT DEFAULT NULL,
`created_at` DATETIME DEFAULT CURRENT_TIMESTAMP,
`attachment_name` VARCHAR(255) DEFAULT NULL,
PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='指南版本';

```

说明：`attachment\_name` 存储上传的标注规范附件文件名。

2.3 一级标签表

```

CREATE TABLE IF NOT EXISTS `label_l1` (
  `id` INT UNSIGNED AUTO_INCREMENT,
  `guide_version_id` INT UNSIGNED NOT NULL,
  `name` VARCHAR(50) NOT NULL,
  `abbr` VARCHAR(30) NOT NULL,
  `description` TEXT DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_version_abbr` (`guide_version_id`, `abbr`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='一级标签';

```

2.4 二级标签表

```

CREATE TABLE IF NOT EXISTS `label_l2` (
  `id` INT UNSIGNED AUTO_INCREMENT,
  `guide_version_id` INT UNSIGNED NOT NULL,
  `parent_l1_id` INT UNSIGNED NOT NULL,
  `name` VARCHAR(50) NOT NULL,
  `abbr` VARCHAR(30) NOT NULL,
  `description` TEXT DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_parent_l1` (`parent_l1_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='二级标签';

```

说明：二级标签通过 `parent\_l1\_id` 关联到一级标签。

2.5 关系类型表

```

CREATE TABLE IF NOT EXISTS `relation_type` (
  `id` INT UNSIGNED AUTO_INCREMENT,
  `guide_version_id` INT UNSIGNED NOT NULL,
  `name` VARCHAR(50) NOT NULL,
  `abbr` VARCHAR(30) NOT NULL,
  `description` TEXT DEFAULT NULL,
  `is_binary` TINYINT NOT NULL DEFAULT 1,
  PRIMARY KEY (`id`),

```

```
    UNIQUE KEY `uk_version_relation_abbr` (`guide_version_id`, `abbr`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='关系类型';
```

2.6 全局文书库表

```
CREATE TABLE IF NOT EXISTS `global_document` (
  `id` BIGINT UNSIGNED AUTO_INCREMENT,
  `title` VARCHAR(255) NOT NULL,
  `file_name` VARCHAR(255) NOT NULL DEFAULT '',
  `file_type` VARCHAR(255) NOT NULL DEFAULT '' COMMENT '文书类型，用户自定义，如
民事判决书',
  `extracted_text` LONGTEXT NOT NULL,
  `uploaded_at` DATETIME DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='全局文书库';
```

2.7 任务表

```
CREATE TABLE IF NOT EXISTS `task` (
  `id` INT AUTO_INCREMENT,
  `title` VARCHAR(100) NOT NULL,
  `description` TEXT DEFAULT NULL,
  `status` ENUM('标注中', '待裁定', '可导出') NOT NULL DEFAULT '标注中',
  `creator_id` BIGINT UNSIGNED NOT NULL,
  `guide_version_id` INT UNSIGNED DEFAULT NULL,
  `created_at` DATETIME DEFAULT CURRENT_TIMESTAMP,
  `stage_changed_at` DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  KEY `idx_creator_id` (`creator_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

说明：任务状态使用 ENUM 类型，支持三种状态：标注中 → 待裁定 → 可导出。

2.8 任务成员表

```
CREATE TABLE IF NOT EXISTS `task_member` (
  `id` INT AUTO_INCREMENT,
  `task_id` INT NOT NULL,
  `user_id` BIGINT UNSIGNED NOT NULL,
  `role_in_task` ENUM('标注员', '裁定者') NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_task_user_role` (`task_id`, `user_id`, `role_in_task`),
  KEY `idx_user_id` (`user_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

说明：任务成员不按文档粒度分配，标注员对整个任务的所有文档负责。

2.9 任务文档表

```
CREATE TABLE IF NOT EXISTS `task_document` (  
  `id` INT AUTO_INCREMENT,  
  `task_id` INT NOT NULL,  
  `source_type` ENUM('UPLOAD', 'RECREATE', 'GLOBAL') NOT NULL DEFAULT 'GLOBAL',  
  `global_doc_id` BIGINT UNSIGNED DEFAULT NULL,  
  `file_name` VARCHAR(255) NOT NULL,  
  `file_path` VARCHAR(500) DEFAULT NULL,  
  `extracted_text` LONGTEXT DEFAULT NULL,  
  `uploaded_at` DATETIME DEFAULT CURRENT_TIMESTAMP,  
  `status` VARCHAR(20) NOT NULL DEFAULT '标注中' COMMENT '文书阶段：标注中/待裁定/可导出',  
  PRIMARY KEY (`id`),  
  KEY `idx_task_id` (`task_id`),  
  KEY `idx_global_doc_id` (`global_doc_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

说明：`source_type` 支持三种来源：`GLOBAL`（引用全局文书库）、`UPLOAD`（本地上传）、`RECREATE`（重新创建）。

2.10 标注结果表

```
CREATE TABLE IF NOT EXISTS `annotation` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  `task_id` BIGINT UNSIGNED NOT NULL,  
  `document_id` BIGINT UNSIGNED NOT NULL,  
  `user_id` BIGINT UNSIGNED NOT NULL,  
  `record_type` VARCHAR(20) NOT NULL DEFAULT 'ANNOTATION' COMMENT 'ANNOTATION=标注员提交, ARBITRATION=裁定结果',  
  `status` VARCHAR(20) NOT NULL DEFAULT 'DRAFT',  
  `is_final` TINYINT NOT NULL DEFAULT 0,  
  `guide_version_id` BIGINT UNSIGNED DEFAULT NULL,  
  `guide_snapshot` JSON DEFAULT NULL,  
  `submitted_at` DATETIME DEFAULT NULL,  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP,  
  `layout_json` JSON DEFAULT NULL COMMENT '论证图布局覆盖',  
  `reject_reason` VARCHAR(500) DEFAULT NULL COMMENT '裁定不予采纳理由',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_annotation_task_doc_user_type` (`task_id`, `document_id`,  
  `user_id`, `record_type`),  
  KEY `idx_annotation_task` (`task_id`),  
  KEY `idx_annotation_doc` (`document_id`),  
  KEY `idx_annotation_user` (`user_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='标注结果表';
```

说明：

- `record_type` 区分标注员提交（`ANNOTATION`）和裁定结果（`ARBITRATION`）。
- `guide_snapshot` 存储标注时使用的标签配置快照（JSON 格式）。
- `layout_json` 存储论证图的节点布局坐标。
- `reject_reason` 存储裁定退回原因。

2.11 命题表

```
CREATE TABLE IF NOT EXISTS `proposition` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  `annotation_id` BIGINT UNSIGNED NOT NULL,  
  `display_id` VARCHAR(20) NOT NULL,  
  `sequence_no` INT NOT NULL,  
  `start_pos` INT NOT NULL,  
  `end_pos` INT NOT NULL,  
  `selected_text` TEXT NOT NULL,  
  `label_l1` VARCHAR(10) NOT NULL,  
  `label_l2` VARCHAR(30) DEFAULT NULL,  
  `label_path` VARCHAR(50) NOT NULL,  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_prop_annotation_display` (`annotation_id`, `display_id`),  
  UNIQUE KEY `uk_prop_annotation_sequence` (`annotation_id`, `sequence_no`),  
  KEY `idx_prop_annotation_range` (`annotation_id`, `start_pos`, `end_pos`),  
  CONSTRAINT `fk_prop_annotation`  
    FOREIGN KEY (`annotation_id`) REFERENCES `annotation` (`id`)  
    ON DELETE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='命题表';
```

说明：

- 命题通过 `annotation_id` 关联到标注记录，而非直接关联文档。
- `display_id` 是前端展示用的命题标识（如 P1、P2）。
- `label_l1` / `label_l2` 冗余存储标签简称，便于快速读取。
- `label_path` 存储完整标签路径（如 GM-L）。

2.12 关系表

```
CREATE TABLE IF NOT EXISTS `argument_relation` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  `annotation_id` BIGINT UNSIGNED NOT NULL,  
  `display_id` VARCHAR(20) NOT NULL,  
  `sequence_no` INT NOT NULL,  
  `relation_type` VARCHAR(10) NOT NULL,  
  `expression` VARCHAR(500) DEFAULT NULL,  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```

`updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
PRIMARY KEY (`id`),
UNIQUE KEY `uk_rel_annotation_display` (`annotation_id`, `display_id`),
UNIQUE KEY `uk_rel_annotation_sequence` (`annotation_id`, `sequence_no`),
KEY `idx_rel_annotation_type` (`annotation_id`, `relation_type`),
CONSTRAINT `fk_rel_annotation`
    FOREIGN KEY (`annotation_id`) REFERENCES `annotation` (`id`)
    ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='关系表';

```

说明:

- 关系表名为 `argument_relation`，通过 `annotation_id` 关联标注记录。
- 关系的嵌套通过 `relation_member` 表实现（`member_type='R'` 时引用其他关系）。
- `expression` 字段存储关系的自然语言表达式。

2.13 关系成员明细表

```

CREATE TABLE IF NOT EXISTS `relation_member` (
    `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    `relation_id` BIGINT UNSIGNED NOT NULL,
    `member_type` VARCHAR(1) NOT NULL,
    `proposition_id` BIGINT UNSIGNED DEFAULT NULL,
    `child_relation_id` BIGINT UNSIGNED DEFAULT NULL,
    `member_role` VARCHAR(10) NOT NULL DEFAULT 'MEMBER',
    `member_order` INT NOT NULL DEFAULT 1,
    `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (`id`),
    KEY `idx_member_relation` (`relation_id`),
    KEY `idx_member_relation_order` (`relation_id`, `member_order`),
    KEY `idx_member_prop_ref` (`proposition_id`),
    KEY `idx_member_rel_ref` (`child_relation_id`),
    CONSTRAINT `fk_member_relation`
        FOREIGN KEY (`relation_id`) REFERENCES `argument_relation` (`id`)
        ON DELETE CASCADE,
    CONSTRAINT `fk_member_prop`
        FOREIGN KEY (`proposition_id`) REFERENCES `proposition` (`id`)
        ON DELETE CASCADE,
    CONSTRAINT `fk_member_child_relation`
        FOREIGN KEY (`child_relation_id`) REFERENCES `argument_relation` (`id`)
        ON DELETE CASCADE,
    CONSTRAINT `chk_member_ref` CHECK (
        (`member_type` = 'P' AND `proposition_id` IS NOT NULL AND
`child_relation_id` IS NULL)
        OR
        (`member_type` = 'R' AND `proposition_id` IS NULL AND `child_relation_id`
IS NOT NULL)
    )
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='关系成员表';

```

设计要点：

- `member_type` 为 P 时引用命题 (`proposition_id`)，为 R 时引用子关系 (`child_relation_id`)。
- `CHECK` 约束确保两种类型互斥，保证数据完整性。
- 支持关系嵌套：关系可包含子关系作为成员。

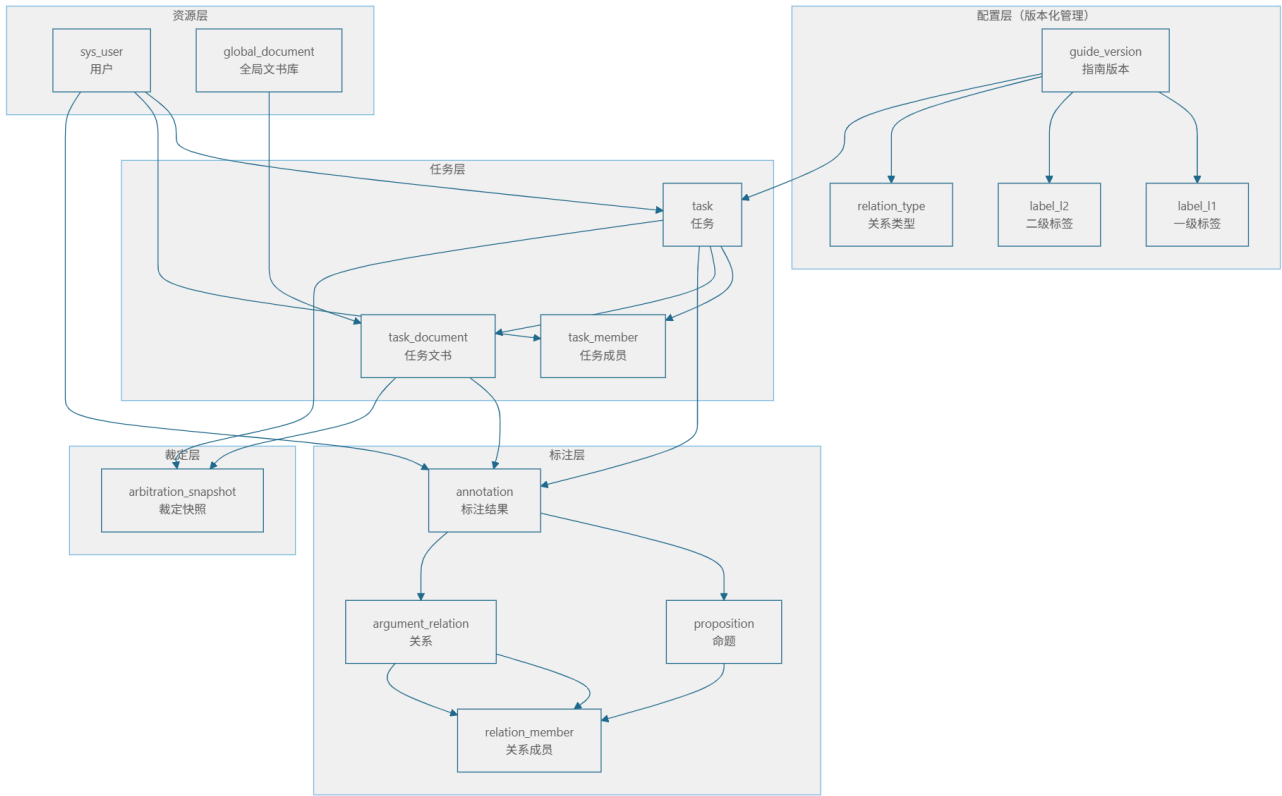
2.14 裁定快照表

```
CREATE TABLE IF NOT EXISTS `arbitration_snapshot` (  
  `id` INT AUTO_INCREMENT,  
  `task_id` INT NOT NULL,  
  `task_document_id` INT NOT NULL,  
  `arbitrator_id` BIGINT UNSIGNED NOT NULL,  
  `adopted_from` VARCHAR(50) DEFAULT NULL,  
  `final_result` TINYINT NOT NULL DEFAULT 0,  
  `arbitrated_at` DATETIME DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_task_doc` (`task_id`, `task_document_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='裁定快照';
```

说明：

- 每个文档只有一条最终裁定记录 (UNIQUE `uk_task_doc`)。
- `adopted_from` 记录采纳来源 (如 `annotator_2` 表示采纳了标注员2的结果，或 `manual` 表示手动裁定)。
- 最终的命题和关系数据存储在 `annotation` 表中 (`record_type='ARBITRATION'`)。

3. 数据存储架构



Mermaid 源文件：[P3-数据存储架构.mmd](#)

4. 索引设计表

表名	索引名称	索引字段	索引类型	设计原因
sys_user	PRIMARY	id	主键	唯一标识每条记录
sys_user	uk_username	username	唯一索引	登录时根据用户名查询，要求唯一且快速定位
guide_version	PRIMARY	id	主键	唯一标识每条记录
label_l1	PRIMARY	id	主键	唯一标识每条记录
label_l1	uk_version_abbr	(guide_version_id, abbr)	唯一复合	保证同一版本下一级标签简称唯一

表名	索引名称	索引字段	索引类型	设计原因
			索引	
label_l2	PRIMARY	id	主键	唯一标识每条记录
label_l2	idx_parent_l1	parent_l1_id	普通索引	查询某个一级标签下的所有二级标签
relation_type	PRIMARY	id	主键	唯一标识每条记录
relation_type	uk_version_relation_abbr	(guide_version_id, abbr)	唯一复合索引	保证关系类型简称唯一；支持按版本快速加载
global_document	PRIMARY	id	主键	唯一标识每条记录
task	PRIMARY	id	主键	唯一标识每条记录
task	idx_creator_id	creator_id	普通索引	查询某个创建者创建的所有任务
task_member	PRIMARY	id	主键	唯一标识每条记录
task_member	uk_task_user_role	(task_id, user_id, role_in_task)	唯一复合索引	防止重复分配；同时覆盖查询某任务的所有成员
task_member	idx_user_id	user_id	普通索引	查询用户参与的所有任务

表名	索引名称	索引字段	索引类型	设计原因
task_document	PRIMARY	id	主键	唯一标识每条记录
task_document	idx_task_id	task_id	普通索引	核心索引： 查询某个任务下的所有数据条目
task_document	idx_global_doc_id	global_doc_id	普通索引	查询某个全局文件被哪些任务引用
annotation	PRIMARY	id	主键	唯一标识每条记录
annotation	uk_annotation_task_doc_user_type	(task_id, document_id, user_id, record_type)	唯一复合索引	保证每个用户对每个文档每种记录类型仅一条
annotation	idx_annotation_task	task_id	普通索引	按任务批量加载所有标注结果（裁决比对）
annotation	idx_annotation_doc	document_id	普通索引	按文档查询标注结果
annotation	idx_annotation_user	user_id	普通索引	查询某个用户的所有标注记录
proposition	PRIMARY	id	主键	唯一标识每条记录
proposition	uk_prop_annotation_display	(annotation_id, display_id)	唯一复合	同一标注记录内 display_id 唯一

表名	索引名称	索引字段	索引类型	设计原因
			索引	
proposition	uk_prop_annotation_sequence	(annotation_id, sequence_no)	唯一复合索引	支持按序号排序和重排
proposition	idx_prop_annotation_range	(annotation_id, start_pos, end_pos)	复合索引	按文本位置范围查询命题
argument_relation	PRIMARY	id	主键	唯一标识每条记录
argument_relation	uk_rel_annotation_display	(annotation_id, display_id)	唯一复合索引	同一标注记录内 display_id 唯一
argument_relation	uk_rel_annotation_sequence	(annotation_id, sequence_no)	唯一复合索引	支持按序号排序和重排
argument_relation	idx_rel_annotation_type	(annotation_id, relation_type)	复合索引	按关系类型筛选
relation_member	PRIMARY	id	主键	唯一标识每条记录
relation_member	idx_member_relation	relation_id	普通索引	查询某个关系的所有成员

表名	索引名称	索引字段	索引类型	设计原因
relation_member	idx_member_relation_order	(relation_id, member_order)	复合索引	按顺序加载关系成员
relation_member	idx_member_prop_ref	proposition_id	普通索引	反向查询： 查找哪些关系引用了某个命题
relation_member	idx_member_rel_ref	child_relation_id	普通索引	反向查询： 查找哪些关系引用了某个子关系
arbitration_snapshot	PRIMARY	id	主键	唯一标识每条记录
arbitration_snapshot	uk_task_doc	(task_id, task_document_id)	唯一复合索引	确保每个文档仅一条最终裁决结果